

任务 1：知识图谱本体推理（作业）

基于 Jena 的规则引擎实现推理

本实践任务要求每位同学独立完成提交

本作业要求使用 Java 语言开发，开发环境任意

关于在 Java 项目中使用 Jena 库的配置布置，请自行查阅文档完成

1. 规则引擎

Jena 包含了一个通用的规则推理机。推理机支持在 RDF 图上推理，提供前向链、后向链和二者混合执行模式。可以通过 GenericRuleReasoner 来进行配置参数，使用各种推理引擎。要使用 GenericRuleReasoner，需要一个规则集来定义其行为。

2. Rule 的语法与结构

规则通过 Rule 对象来进行定义，包含 body terms 列表 (premises), head terms 列表 (conclusions) 和可选的 name 和可选的 direction。

```
_Rule_      :=  _bare-rule_ .
               or  [ _bare-rule_ ]
               or  [ ruleName : _bare-rule_ ]

_bare-rule_ :=  _term_, ... _term_ -> _hterm_, ... _hterm_  // forward rule
               or  _bhterm_ <- _term_, ... _term_          // backward rule

_hterm      :=  term
_ or [ _bare-rule_ ]

_term_      :=  (_node_, _node_, _node_)                  // triple pattern
               or  (_node_, _node_, _functor_)             // extended triple pattern
               or  builtin(_node_, ... _node_)             // invoke procedural primitive

_bhterm_    :=  (_node_, _node_, _node_)                  // triple pattern

_functor_   :=  functorName(_node_, ... _node_)           // structured literal

_node_      :=  _uri-ref_                                  // e.g. http://foo.com/eg
               or  prefix:localname                       // e.g. rdf:type
               or  <_uri-ref>                             // e.g. <myscheme:myuri>
               or  ?_varname_ // variable
               or  'a literal'                             // a plain string literal
               or  'lex'^^typeURI                         // a typed literal, xsd:* type names supported
               or  number                                  // e.g. 42 or 25.5
```

3. Demo1：喜剧演员

例如，在一个电影知识图谱里，如果一个演员参演的电影的类型是喜剧片，我们可以认为这个演员是喜剧演员。

推理规则

```
[ruleComedian: (?p :hasActedIn ?m) (?m :hasGenre ?g) (?g :genreName '喜剧') -> (?p rdf:type :Comedian)]
```

用代码来实现：

```
String prefix = "http://www.test.com/kg/#";

Graph data = GraphMemFactory.createGraphMem();

// 定义节点

Node movie = NodeFactory.createURI(prefix + "movie");

Node hasActedIn = NodeFactory.createURI(prefix + "hasActedIn");

Node hasGenre = NodeFactory.createURI(prefix + "hasGenre");

Node genreName = NodeFactory.createURI(prefix + "genreName");

Node genre = NodeFactory.createURI(prefix + "genre");

Node person = NodeFactory.createURI(prefix + "person");

Node Comedian = NodeFactory.createURI(prefix + "Comedian");

// 添加三元组

data.add(Triple.create(genre, genreName, NodeFactory.createLiteralString("喜剧")));

data.add(Triple.create(movie, hasGenre, genre));

data.add(Triple.create(person, hasActedIn, movie));

// 创建推理机

GenericRuleReasoner reasoner = (GenericRuleReasoner) GenericRuleReasonerFactory.theInstance().create(null);

PrintUtil.registerPrefix("", prefix);

// 设置规则

// 在一个电影知识图谱里，如果一个演员参演的电影的类型是喜剧片，我们可以认为这个演员是喜剧演员

// 推理规则：

// [ruleComedian: (?p :hasActedIn ?m) (?m :hasGenre ?g) (?g :genreName '喜剧') -> (?p rdf:type :Comedian)]

reasoner.setRules(Rule.parseRules(

    "[ruleComedian: (?p :hasActedIn ?m) (?m :hasGenre ?g) (?g :genreName '喜剧') -> (?p rdf:type :Comedian)] \\n"

    + "-> tableAll()."););

reasoner.setMode(GenericRuleReasoner.HYBRID); // HYBRID 混合推理

InfGraph infgraph = reasoner.bind(data);

infgraph.setDerivationLogging(true);

// 执行推理

Iterator<Triple> tripleIterator = infgraph.find(person, null, null);

while (tripleIterator.hasNext()) {

    System.out.println(PrintUtil.print(tripleIterator.next()));

}
```

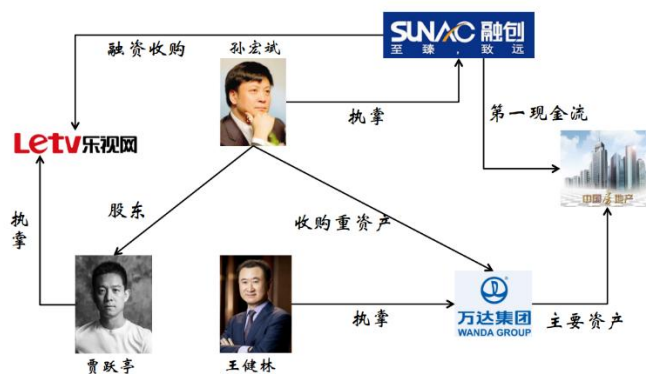
输出结果：

```
(:person rdf:type :Comedian)
(:person :hasActedIn :movie)
```

可以看到，已经给 person 加上了 Comedian。

4. Demo2: 关联交易

公司交易图谱：



推理任务：

- (1) 执掌一家公司就一定是这家公司的股东；
- (2) 某人同时是两家公司的股东，那么这两家公司一定有关联交易；

首先，构造好图谱，为了方便理解，用中文变量：

```
Model myMod = ModelFactory.createDefaultModel();

String finance = "http://www.example.org/kse/finance#";

Resource 孙宏斌 = myMod.createResource(finance + "孙宏斌");
Resource 融创中国 = myMod.createResource(finance + "融创中国");
Resource 乐视网 = myMod.createResource(finance + "乐视网");
Property 执掌 = myMod.createProperty(finance + "执掌");
Resource 贾跃亭 = myMod.createResource(finance + "贾跃亭");
Resource 地产公司 = myMod.createResource(finance + "地产公司");
Resource 公司 = myMod.createResource(finance + "公司");
Resource 法人实体 = myMod.createResource(finance + "法人实体");
Resource 人 = myMod.createResource(finance + "人");
Property 主要收入 = myMod.createProperty(finance + "主要收入");
Resource 地产事业 = myMod.createResource(finance + "地产事业");
Resource 王健林 = myMod.createResource(finance + "王健林");
Resource 万达集团 = myMod.createResource(finance + "万达集团");
```

```

Property 主要资产 = myMod.createProperty(finance + "主要资产");

Property 股东 = myMod.createProperty(finance + "股东");
Property 关联交易 = myMod.createProperty(finance + "关联交易");
Property 收购 = myMod.createProperty(finance + "收购");

// 加入三元组
myMod.add(孙宏斌, 执掌, 融创中国);
myMod.add(贾跃亭, 执掌, 乐视网);
myMod.add(王健林, 执掌, 万达集团);
myMod.add(乐视网, RDF.type, 公司);
myMod.add(万达集团, RDF.type, 公司);
myMod.add(融创中国, RDF.type, 地产公司);
myMod.add(地产公司, RDFS.subClassOf, 公司);
myMod.add(公司, RDFS.subClassOf, 法人实体);
myMod.add(孙宏斌, RDF.type, 人);
myMod.add(贾跃亭, RDF.type, 人);
myMod.add(王健林, RDF.type, 人);
myMod.add(万达集团, 主要资产, 地产事业);
myMod.add(融创中国, 主要收入, 地产事业);
myMod.add(孙宏斌, 股东, 乐视网);
myMod.add(孙宏斌, 收购, 万达集团);

PrintUtil.registerPrefix("", finance);

// 输出当前模型
StmtIterator i = myMod.listStatements(null, null, (RDFNode) null);
while (i.hasNext()) {
    System.out.println(" - " + PrintUtil.print(i.nextStatement()));
}

```

上图所示的图谱，包含如下的三元组，执行上述代码会输出：

- (:公司 rdfs:subClassOf :法人实体)
- (:万达集团 :主要资产 :地产事业)
- (:万达集团 rdf:type :公司)
- (:地产公司 rdfs:subClassOf :公司)
- (:融创中国 :主要收入 :地产事业)
- (:融创中国 rdf:type :地产公司)
- (:孙宏斌 :股东 :乐视网)
- (:孙宏斌 rdf:type :人)
- (:孙宏斌 :执掌 :融创中国)
- (:乐视网 rdf:type :公司)
- (:贾跃亭 rdf:type :人)
- (:贾跃亭 :执掌 :乐视网)
- (:王健林 rdf:type :人)
- (:王健林 :执掌 :万达集团)

定义推理规则:

- (1) 执掌一家公司就一定是这家公司的股东
- (2) 收购一家公司，就是这家公司的股东
- (3) 某人同时是两家公司的股东，那么这两家公司一定有关联交易

用 Jena 规则来表示:

```
[ruleHoldShare: (?p :执掌 ?c) -> (?p :股东 ?c)]
[ruleHoldShare2: (?p :收购 ?c) -> (?p :股东 ?c)]
[ruleConnTrans: (?p :股东 ?c) (?p :股东 ?c2) -> (?c :关联交易 ?c2)]
```

在前面代码之后追加如下代码:

```
// 执行推理:
GenericRuleReasoner reasoner = (GenericRuleReasoner) GenericRuleReasonerFactory.theInstance().create(null);
reasoner.setRules(Rule.parseRules(
    "[ruleHoldShare: (?p :执掌 ?c) -> (?p :股东 ?c)] \n"
    + "[ruleHoldShare2: (?p :收购 ?c) -> (?p :股东 ?c)] \n"
    + "[ruleConnTrans: (?p :股东 ?c) (?p :股东 ?c2) -> (?c :关联交易 ?c2)] \n"
    + "-> tableAll()."));
reasoner.setMode(GenericRuleReasoner.HYBRID);

InfGraph infgraph = reasoner.bind(myMod.getGraph());
infgraph.setDerivationLogging(true);

System.out.println("推理后...\n");
```

```

Iterator<Triple> tripleIterator = infgraph.find(null, null, null);
while (tripleIterator.hasNext()) {
    System.out.println(" - " + PrintUtil.print(tripleIterator.next()));
}

```

输出结果：

推理后...

- (: 万达集团 : 关联交易 : 乐视网)
- (: 万达集团 : 关联交易 : 融创中国)
- (: 万达集团 : 关联交易 : 万达集团)
- (: 孙宏斌 : 股东 : 万达集团)
- (: 孙宏斌 : 股东 : 融创中国)
- (: 融创中国 : 关联交易 : 万达集团)
- (: 融创中国 : 关联交易 : 乐视网)
- (: 融创中国 : 关联交易 : 融创中国)
- (: 乐视网 : 关联交易 : 万达集团)
- (: 乐视网 : 关联交易 : 融创中国)
- (: 乐视网 : 关联交易 : 乐视网)
- (: 贾跃亭 : 股东 : 乐视网)
- (: 王健林 : 股东 : 万达集团)
- (: 公司 rdfs:subClassOf : 法人实体)
- (: 万达集团 : 主要资产 : 地产事业)
- (: 万达集团 rdf:type : 公司)
- (: 地产公司 rdfs:subClassOf : 公司)
- (: 融创中国 : 主要收入 : 地产事业)
- (: 融创中国 rdf:type : 地产公司)
- (: 孙宏斌 : 收购 : 万达集团)
- (: 孙宏斌 : 股东 : 乐视网)
- (: 孙宏斌 rdf:type : 人)
- (: 孙宏斌 : 执掌 : 融创中国)
- (: 乐视网 rdf:type : 公司)
- (: 贾跃亭 rdf:type : 人)
- (: 贾跃亭 : 执掌 : 乐视网)
- (: 王健林 rdf:type : 人)
- (: 王健林 : 执掌 : 万达集团)

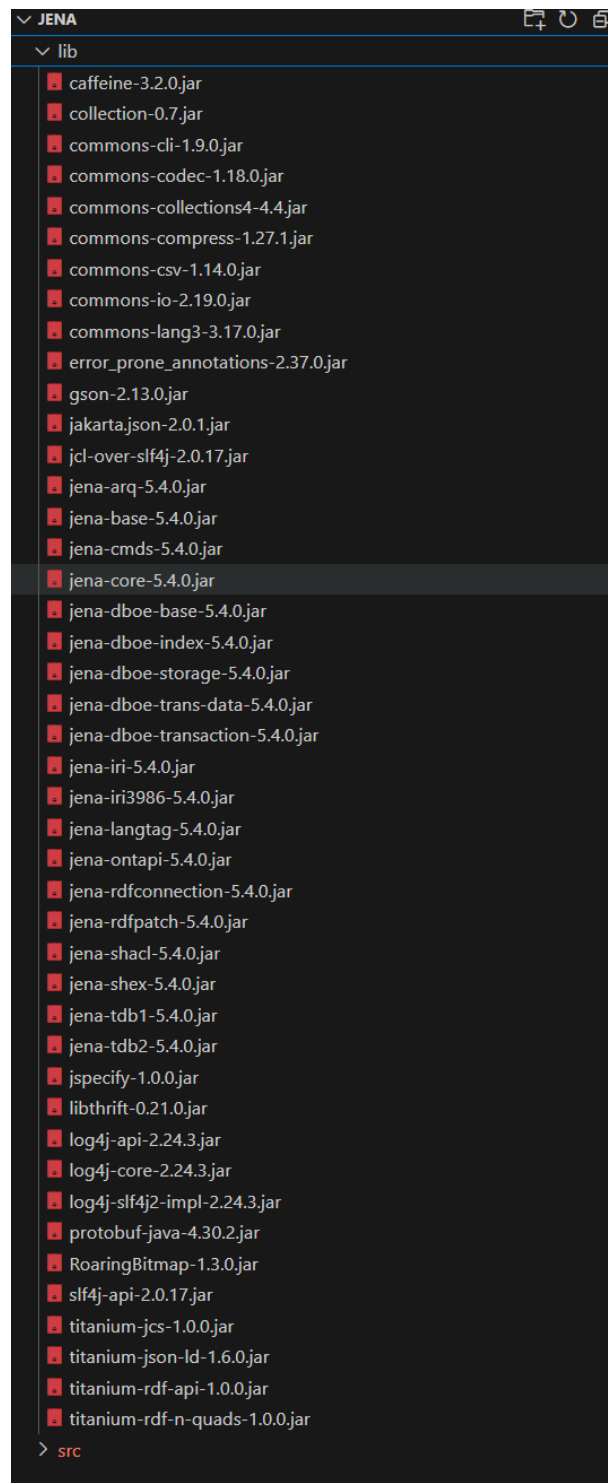
我们看到，推理后孙宏斌是三家公司的股东，三家公司都有关联交易。

【要求】：

【Java 程序项目源代码文件夹打包，上传到智慧树】

【Demo1：程序执行成功的截图，要求有开发环境和程序结果输出】

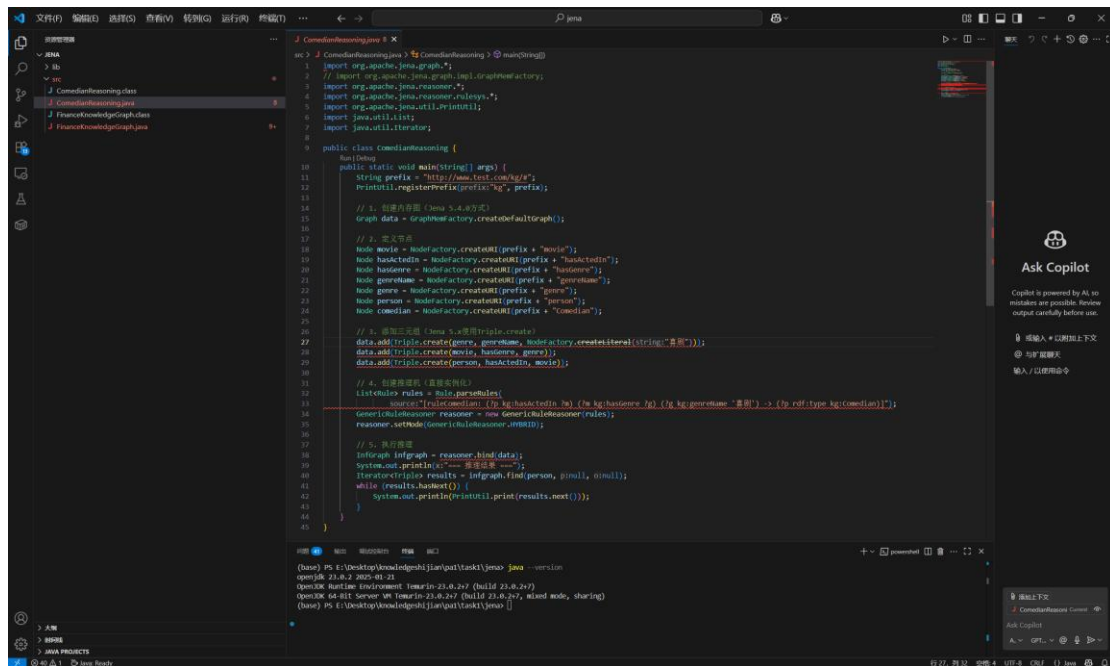
将下载的 apache-jena-5.4.0 下的 lib 放到项目目录中，即为所需要的依赖。



Jdk 版本为 23.0.2

```
问题 41 输出 调试控制台 终端 端口
(base) PS E:\Desktop\knowledgeshijian\pa1\task1\jena> java --version
openjdk 23.0.2 2025-01-21
OpenJDK Runtime Environment Temurin-23.0.2+7 (build 23.0.2+7)
OpenJDK 64-Bit Server VM Temurin-23.0.2+7 (build 23.0.2+7, mixed mode, sharing)
(base) PS E:\Desktop\knowledgeshijian\pa1\task1\jena>
```

在 src 中编写 demo1:



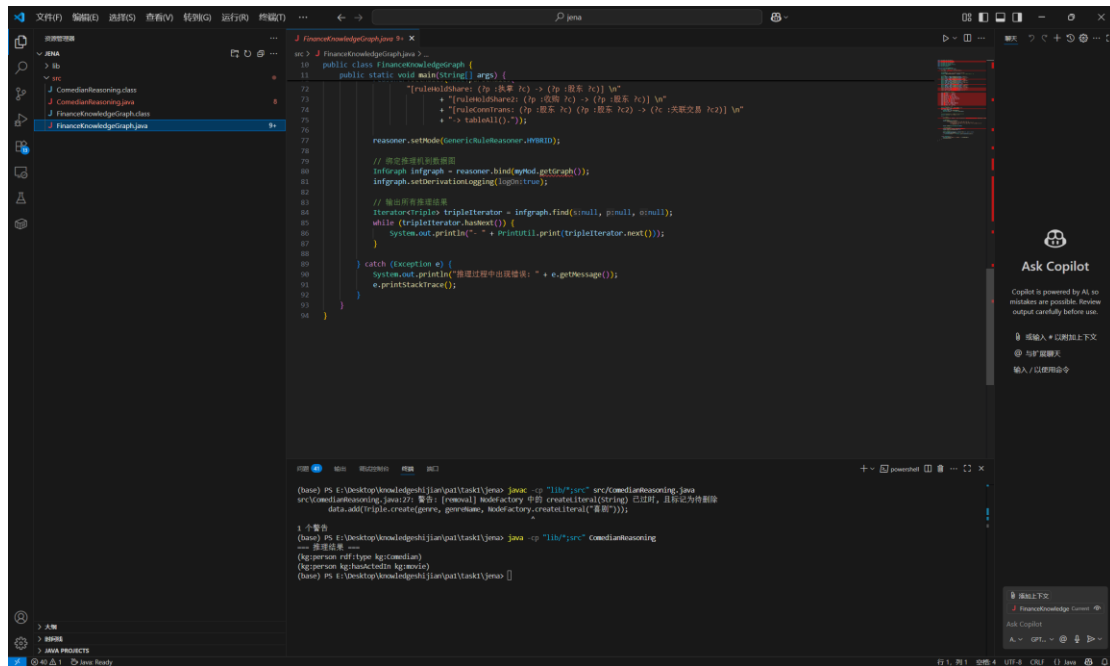
```
src > J ComedianReasoning.java > src\ComedianReasoning.java
1 import org.apache.jena.graph.*;
2 // import org.apache.jena.graph.impl.GraphFactory;
3 import org.apache.jena.reasoner.*;
4 import org.apache.jena.reasoner.rulesys.*;
5 import org.apache.jena.util.iterator.*;
6 import java.util.Iterator;
7
8 public class ComedianReasoning {
9     public static void main(String[] args) {
10         String prefix = "http://www.test.com/kg/#";
11         PrintUtil.registerPrefix(prefix, "kg");
12
13         // 1. 创建内存型 Jena 3.4.0 方式
14         Graph data = GraphFactory.createDefaultGraph();
15
16         // 2. 定义节点
17         Node genre = NodeFactory.createURI(prefix + "genre");
18         Node hasActedIn = NodeFactory.createURI(prefix + "hasActedIn");
19         Node genreName = NodeFactory.createURI(prefix + "hasGenre");
20         Node genreName = NodeFactory.createURI(prefix + "genreName");
21         Node person = NodeFactory.createURI(prefix + "person");
22         Node comedian = NodeFactory.createURI(prefix + "comedian");
23
24         // 3. 添加三元组 (Jena 3.4.0 使用 Triple.create)
25         data.add(Triple.create(genre, genreName, NodeFactory.createLiteral("喜剧")));
26         data.add(Triple.create(person, hasActedIn, NodeFactory.createLiteral("喜剧")));
27         data.add(Triple.create(person, hasActedIn, NodeFactory.createLiteral("喜剧")));
28
29         // 4. 设置推理引擎 (设置推理规则)
30         List<Rule> rules = RuleParser.parseRules(
31             "rules: { ruleComedian: ( ?p kg:hasActedIn ?a ) ( ?p kg:hasGenre ?g ) ( ?p kg:genreName ?g ) -> ( ?p rdf:type kg:comedian ) }";
32         );
33         GenericRuleReasoner reasoner = new GenericRuleReasoner(rules);
34         reasoner.setBase(GraphFactory.createDefaultGraph());
35
36         // 5. 执行推理
37         InferenceGraph graph = reasoner.find(data);
38         System.out.println("=== 推理结果 ===");
39         Iterator<Triple> results = graph.find(person, 0, null, 0, null);
40         while (results.hasNext()) {
41             System.out.println(PrintUtil.print(results.next()));
42         }
43     }
44 }
```

输入 `javac -cp "lib/*;src" src/ComedianReasoning.java` 进行编译, 输入 `java -cp "lib/*;src" ComedianReasoning` 运行程序, 得到的结果为:

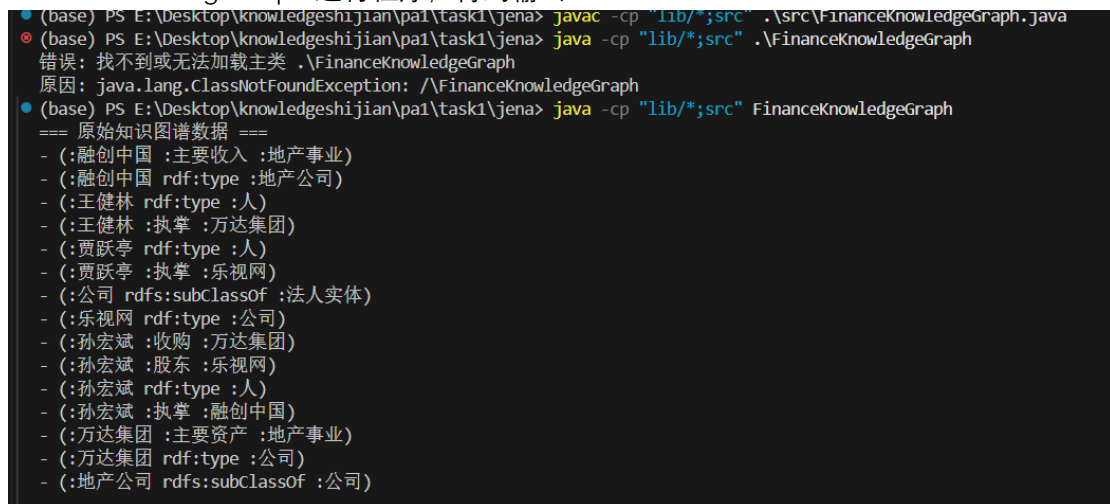
```
(base) PS E:\Desktop\knowledgeshijian\pa1\task1\jena> javac -cp "lib/*;src" src/ComedianReasoning.java
src\ComedianReasoning.java:27: 警告: [removal] NodeFactory 中的 createLiteral(String) 已过时, 且标记为待删除
    data.add(Triple.create(genre, genreName, NodeFactory.createLiteral("喜剧")));
                                     ^
1 个警告
(base) PS E:\Desktop\knowledgeshijian\pa1\task1\jena> java -cp "lib/*;src" ComedianReasoning
=== 推理结果 ===
(kg:person rdf:type kg:Comedian)
(kg:person kg:hasActedIn kg:movie)
(base) PS E:\Desktop\knowledgeshijian\pa1\task1\jena>
```

【Demo2: 程序执行成功的截图, 要求有开发环境和程序结果输出】

在 src 中编写程序:



输入 `javac -cp "lib/*;src" .\src\FinanceKnowledgeGraph.java` 进行编译, 输入 `java -cp "lib/*;src" FinanceKnowledgeGraph` 运行程序, 得到输出:



```
=== 推理结果 ===
- (:融创中国 :关联交易 :融创中国)
- (:融创中国 :关联交易 :万达集团)
- (:融创中国 :关联交易 :乐视网)
- (:王健林 :股东 :万达集团)
- (:贾跃亭 :股东 :乐视网)
- (:乐视网 :关联交易 :融创中国)
- (:乐视网 :关联交易 :万达集团)
- (:乐视网 :关联交易 :乐视网)
- (:孙宏斌 :股东 :融创中国)
- (:孙宏斌 :股东 :万达集团)
- (:万达集团 :关联交易 :融创中国)
- (:万达集团 :关联交易 :万达集团)
- (:万达集团 :关联交易 :乐视网)
- (:融创中国 :主要收入 :地产事业)
- (:融创中国 rdf:type :地产公司)
- (:王健林 rdf:type :人)
- (:王健林 :执掌 :万达集团)
- (:贾跃亭 rdf:type :人)
- (:贾跃亭 :执掌 :乐视网)
- (:公司 rdfs:subClassOf :法人实体)
- (:乐视网 rdf:type :公司)
- (:孙宏斌 :收购 :万达集团)
- (:孙宏斌 :股东 :乐视网)
- (:孙宏斌 rdf:type :人)
- (:孙宏斌 :执掌 :融创中国)
- (:万达集团 :主要资产 :地产事业)
- (:万达集团 rdf:type :公司)
- (:地产公司 rdfs:subClassOf :公司)
❖ (base) PS E:\Desktop\knowledgeshijian\pa1\task1\jena> |
```